

Mini Hiring Pipeline — Architecture & Design Document

GitHub Repository: https://github.com/Asif68113156/HIRE_FLOW

Author: Asif Siddiqui | **Tech Stack:** Python 3.11+, Flask, SQLAlchemy, RapidFuzz, Bootstrap 5

1. Executive Summary & Problem Statement

Recruitment teams frequently struggle with fragmented data, manual stage tracking, and rigid database queries. The **Mini Hiring Pipeline** provides an enterprise-grade candidate management platform built around strict backend pipeline validation, an immutable audit trail, and an intelligent natural language search engine.

2. System Architecture & 3-Tier Design

The system enforces a clean 3-tier modular architecture:

- **Presentation Layer:** Flask Controllers (routes/candidates.py, routes/search.py) rendering Jinja2 templates.
- **Service & Business Layer:** Centralized business logic (services/candidate_service.py, services/transitions.py).
- **Data & Search Layer:** SQLAlchemy Models (models/candidate.py, history.py) and custom NLP module (search/).

3. Data Models & Immutability Guarantee

- **Candidate Model:** Tracks full name, email, phone, role, mandatory resume PDF URL, notes, and current stage.
- **Immutable Stage History Ledger:** An append-only table (stage_history) logging candidate stage transitions (from_stage, to_stage, UTC timestamp, actor). History records are read-only and cannot be altered or deleted.

4. Server-Side Stage Transition Logic

Stage progression follows a strict state machine: **Applied** → **Screening** → **Interview** → **Offer** → **Hired** (with **Rejected** allowed from any active stage).

Validation is enforced server-side in services/transitions.py. Attempting an out-of-order transition (e.g. *Applied* → *Offer*) throws an InvalidTransitionError returning HTTP 400 Bad Request.

5. Natural Language & Fuzzy Search Engine

- **Query Parser (search/parser.py):** Regex-based extraction of stage names, time in stage (> 7 days), relative dates (Monday), and negative filters (Offer but not Hired).
- **Database Filter (search/filters.py):** Executes efficient ORM queries.
- **Fuzzy String Matcher (search/fuzzy.py):** Uses RapidFuzz (WRatio) to match misspelled candidate names (e.g., 'sharam' → 'Priya Sharma').

6. Senior Engineering Decision: Disagreement with AI

The AI Recommendation: Lock invalid stage transition choices on the client side by disabling HTML dropdown options.

Why I Overrode It: Client-side locking is only a visual preference and easily bypassed by posting direct HTTP payloads. I implemented server-side state validation in services/transitions.py before processing any database commit, guaranteeing strict data integrity.

7. Verified Demo Queries & Performance Matrix

Query	Parsed Intent	Verified Results
sharam	Fuzzy Name Search	Priya Sharma
Interview candidates	Stage: Interview	4 candidates
Screening > 7 days	Time > 7 days in Screening	Aisha, Kiran, Zara
Offer but not Hired	Reached Offer $\wedge$ $\neg$ Hired	Arjun, Vikram, Nikhil
everyone except rejected	Exclude Rejected	13 candidates
Monday	Moved since Monday	12 candidates

8. Project Deliverable Links

GitHub Repository: https://github.com/Asif68113156/HIRE_FLOW

Unit Test Suite: 12 passing integration & unit tests in pytest (0.08s runtime).